GIÁO ÁN FRONTEND
VITE + AXIOS + TEST + RAILS GRAPHQL

Từ cài đặt frontend đến kết nối Rails GraphQL qua Vite Proxy

Mục lục

1. Step 1 — Chuẩn bị Rails backend
2. Step 2 — Kiểm tra Node
3. Step 3 — Tạo Vite React project
4. Step 4 — Cài Axios
5. Step 5 — Tạo cấu trúc frontend
6. Step 6 — Không gọi Rails trực tiếp từ browser trong development
7. Step 7 — Dùng Vite proxy
8. Step 8 — Hiểu proxy
9. Step 9 — baseURL trong Axios
10. Step 10 — Không dùng VITE_API_BASE=http://127.0.0.1:3000 với development proxy
11. Step 11 — Tạo GraphQL client function
12. Step 12 — Tại sao /graphql là đúng
13. Step 13 — Tạo GraphQL query
14. Step 14 — Tạo API getProjects
15. Step 15 — Gọi API trong page
16. Step 16 — CORS error xảy ra vì đâu
17. Step 17 — Kiểm tra request đúng
18. Step 18 — Nếu thấy 502 Bad Gateway
19. Step 19 — Target phải đúng port Rails
20. Step 20 — Restart Vite sau khi sửa config
21. Step 21 — 422 khác 502
22. Step 22 — GraphqlController hiện tại
23. Step 23 — Không vội tắt CSRF
24. Step 24 — Devise session cần cookie
25. Step 25 — CSRF nếu dùng Devise session
26. Step 26 — Proxy /csrf
27. Step 27 — Axios CSRF token
28. Step 28 — Cài Vitest
29. Step 29 — Cài React Testing Library
30. Step 30 — Cấu hình test
31. Step 31 — Setup Testing Library
32. Step 32 — package.json scripts
33. Step 33 — Test component
34. Step 34 — Test API bằng mock
35. Step 35 — Chạy tests
36. Step 36 — Cấu trúc cuối cùng

Step 1 — Chuẩn bị Rails backend

Rails GraphQL backend chạy ở http://127.0.0.1:3000 hoặc http://localhost:3000. Route GraphQL là POST /graphql. GraphqlController nhận query, variables, operationName và truyền current_user vào context.

```
post "/graphql", to: "graphql#execute"

class GraphqlController < ApplicationController
  def execute
    variables = prepare_variables(params[:variables])
    query = params[:query]
    operation_name = params[:operationName]

    context = {
      current_user: current_user
    }

    result = DevelopmentSchema.execute(
      query,
      variables: variables,
      context: context,
      operation_name: operation_name
    )

    render json: result
  end
end
```

Step 2 — Kiểm tra Node

Trước khi cài Vite, kiểm tra Node và npm.

```
node -v
npm -v
```

Step 3 — Tạo Vite React project

Tạo frontend React bằng Vite, cài dependencies và chạy development server. Frontend thường ở localhost:5173 hoặc 5174 nếu port 5173 đang bận.

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

Step 4 — Cài Axios

Axios dùng để gọi Rails API từ frontend.

```
npm install axios
```

Step 5 — Tạo cấu trúc frontend

Tách API client, GraphQL operations, pages, components và tests thay vì đặt tất cả trong App.jsx.

```
frontend/
└── src/
```

```
    ├── api/
    │   ├── axios.js
    │   ├── graphql.js
    │   └── projects.js
    ├── graphql/
    │   ├── queries/
    │   │   └── getProjects.js
    │   └── mutations/
    ├── pages/
    │   └── ProjectsPage.jsx
    ├── components/
    ├── test/
    ├── App.jsx
    └── main.jsx
```

Step 6 — Không gọi Rails trực tiếp từ browser trong development

Nếu Axios dùng baseURL http://127.0.0.1:3000 trong khi frontend ở localhost:5173, browser xem đó là cross-origin request và có thể chặn bởi CORS.

```
// Không nên khi dùng Vite proxy
axios.create({
  baseURL: "http://127.0.0.1:3000"
})
```

Step 7 — Dùng Vite proxy

Vite nhận request /graphql từ browser và forward sang Rails ở port 3000.

```
import { defineConfig } from "vite"
import react from "@vitejs/plugin-react"
import tailwindcss from "@tailwindcss/vite"

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],

  server: {
    proxy: {
      "/graphql": {
        target: "http://127.0.0.1:3000",
        changeOrigin: true,
      },
    },
  },
})
```

Step 8 — Hiểu proxy

Khi frontend gọi /graphql, browser chỉ giao tiếp với Vite. Vite mới là bên forward request sang Rails. Đây là lý do development tránh được CORS khi không gọi Rails URL trực tiếp.

```
Browser
  ↓
http://localhost:5173/graphql
  ↓
Vite Proxy
  ↓
```

```
http://127.0.0.1:3000/graphql
  ↓
Rails
```

Step 9 — baseURL trong Axios

baseURL là địa chỉ gốc Axios ghép với URL của request. Với Vite proxy, để baseURL rỗng để request đi cùng origin frontend.

```
import axios from "axios"

const api = axios.create({
  baseURL: "",
  headers: {
    "Content-Type": "application/json",
  },
  timeout: 10000,
})

export default api
```

Step 10 — Không dùng VITE_API_BASE=http://127.0.0.1:3000 với development proxy

Nếu VITE_API_BASE trỏ thẳng về Rails thì Axios bỏ qua Vite proxy. Trong development proxy, xóa biến đó hoặc để rỗng.

```
VITE_API_BASE=

export const API_BASE_URL =
  import.meta.env.VITE_API_BASE ||
```

Step 11 — Tạo GraphQL client function

graphqlRequest là hàm dùng chung cho mọi query/mutation. Nó gửi operationName, query và variables; đồng thời chuẩn hóa GraphQL errors.

```
import api from "./axios"

export async function graphqlRequest({
  query,
  variables = {},
  operationName,
}) {
  const response = await api.post(
    "/graphql",
    {
      query,
      variables,
      operationName,
    }
  )

  const body = response.data

  if (body.errors?.length) {
    const graphqlError = body.errors[0]
```

```
    const error = new Error(
      graphqlError.message
    )

    error.code =
      graphqlError.extensions?.code

    error.graphqlErrors =
      body.errors

    throw error
  }

  return body.data
}
```

Step 12 — Tại sao /graphql là đúng

Vì baseURL đang rỗng, api.post('/graphql') tạo request tới localhost:5173/graphql. Vite proxy sau đó chuyển sang Rails. Không cần hard-code Rails URL trong graphql.js.

```
api.post("/graphql")

# Browser:
http://localhost:5173/graphql

# Vite forwards to:
http://127.0.0.1:3000/graphql
```

Step 13 — Tạo GraphQL query

Tạo operation GetProjects với pagination variables first và after.

```
export const GET_PROJECTS = `
  query GetProjects(
    $first: Int
    $after: String
  ) {
    projects(
      first: $first
      after: $after
    ) {
      nodes {
        id
        name
      }

      pageInfo {
        hasNextPage
        endCursor
      }
    }
  }
`
```

Step 14 — Tạo API getProjects

API layer gọi graphqlRequest thay vì component tự gọi Axios.

```
import { graphqlRequest } from "./graphql"
import { GET_PROJECTS } from "../graphql/queries/getProjects"

export function getProjects({
  first = 10,
  after = null,
} = {}) {
  return graphqlRequest({
    operationName: "GetProjects",
    query: GET_PROJECTS,
    variables: {
      first,
      after,
    },
  })
}
```

Step 15 — Gọi API trong page

ProjectsPage quản lý loading, success và error state. Component chỉ biết getProjects(), không cần biết Axios/Vite proxy hoạt động thế nào.

```
import { useEffect, useState } from "react"
import { getProjects } from "../api/projects"

function ProjectsPage() {
  const [projects, setProjects] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    async function loadProjects() {
      try {
        const data = await getProjects()
        setProjects(data.projects.nodes)
      } catch (error) {
        setError(error.message)
      } finally {
        setLoading(false)
      }
    }

    loadProjects()
  }, [])

  if (loading) return <p>Loading...</p>
  if (error) return <p>Error: {error}</p>

  return (
    <div>
      <h1>Projects</h1>

      {projects.map((project) => (
        <div key={project.id}>
          {project.name}
        </div>
      ))}
    </div>
  )
}

export default ProjectsPage
```

Step 16 — CORS error xảy ra vì đâu

CORS xảy ra khi browser gọi thẳng Rails ở 127.0.0.1:3000 từ origin localhost:5173. Fix là baseURL rỗng + Vite proxy.

```
Sai:
React -> 127.0.0.1:3000/graphql -> CORS

Đúng:
React -> /graphql -> Vite -> Rails
```

Step 17 — Kiểm tra request đúng

Trong Chrome DevTools > Network > graphql, Request URL phải là localhost:5173/graphql. Nếu vẫn thấy 127.0.0.1:3000/graphql thì frontend vẫn đang hard-code backend URL.

```
Đúng:
http://localhost:5173/graphql

Sai khi dùng Vite proxy:
http://127.0.0.1:3000/graphql
```

Step 18 — Nếu thấy 502 Bad Gateway

502 từ localhost:5173/graphql thường nghĩa Vite đã nhận request nhưng không connect được Rails target. Kiểm tra Rails có chạy không và target có đúng không.

```
bin/rails server
curl http://127.0.0.1:3000/up
```

Step 19 — Target phải đúng port Rails

Nếu Rails chạy port 3000 thì target là :3000. Nếu Rails chạy 3001 thì target phải đổi sang :3001.

```
target: "http://127.0.0.1:3000"

# hoặc nếu Rails chạy 3001:
target: "http://127.0.0.1:3001"
```

Step 20 — Restart Vite sau khi sửa config

Sau khi sửa vite.config.js, restart Vite để chắc chắn proxy config mới được áp dụng.

```
Ctrl+C
npm run dev
```

Step 21 — 422 khác 502

502 nghĩa Vite proxy không kết nối được Rails. 422 nghĩa request đã tới Rails nhưng Rails từ chối xử lý; với Rails + Devise cần kiểm tra CSRF, session hoặc request validation.

```
502 -> proxy/backend connectivity
422 -> Rails đã nhận request nhưng reject
```

Step 22 — GraphqlController hiện tại

GraphqlController kế thừa ApplicationController, nên CSRF protection của Rails có thể áp dụng. Controller vẫn truyền current_user vào GraphQL context.

```
class GraphqlController < ApplicationController
  def execute
    variables = prepare_variables(params[:variables])
    query = params[:query]
    operation_name = params[:operationName]

    context = {
      current_user: current_user
    }

    result = DevelopmentSchema.execute(
      query,
      variables: variables,
      context: context,
      operation_name: operation_name
    )

    render json: result
  end
end
```

Step 23 — Không vội tắt CSRF

Generator GraphQL thường gợi ý protect_from_forgery with: :null_session. Không nên bật ngay nếu bạn muốn current_user đến từ Devise session, vì null_session có thể làm session không còn được dùng như mong muốn.

```
# Không bật vội nếú dùng Devise session:
# protect_from_forgery with: :null_session
```

Step 24 — Devise session cần cookie

Nếu React dùng Devise session, Axios nên gửi credentials/cookies.

```
const api = axios.create({
  baseURL: "",
  withCredentials: true,
  headers: {
    "Content-Type": "application/json",
  },
})
```

Step 25 — CSRF nếu dùng Devise session

Vite app độc lập không có Rails-generated meta csrf-token. Có thể tạo GET /csrf để Rails trả form_authenticity_token cho frontend.

```
class CsrfController < ApplicationController
  def show
    render json: {
      csrf_token: form_authenticity_token
    }
  end
end
```

```
get "/csrf", to: "csrf#show"
```

Step 26 — Proxy /csrf

Nếu dùng endpoint /csrf, proxy luôn path này qua Vite giống /graphql.

```
server: {
  proxy: {
    "/graphql": {
      target: "http://127.0.0.1:3000",
      changeOrigin: true,
    },

    "/csrf": {
      target: "http://127.0.0.1:3000",
      changeOrigin: true,
    },
  },
}
```

Step 27 — Axios CSRF token

Sau khi lấy CSRF token từ Rails, lưu token và thêm X-CSRF-Token vào request headers bằng Axios interceptor.

```
let csrfToken = null

export function setCsrfToken(token) {
  csrfToken = token
}

api.interceptors.request.use((config) => {
  if (csrfToken) {
    config.headers["X-CSRF-Token"] =
      csrfToken
  }

  return config
})
```

Step 28 — Cài Vitest

Vitest dùng để chạy unit/component tests trong Vite project.

```
npm install -D vitest
```

Step 29 — Cài React Testing Library

Cài Testing Library, jest-dom, user-event và jsdom.

```
npm install -D   @testing-library/react   @testing-library/jest-dom   @testing-library/user-event
jsdom
```

Step 30 — Cấu hình test

Thêm cấu hình Vitest vào vite.config.js.

```
test: {
  environment: "jsdom",
  setupFiles: [
    "./src/test/setup.js",
  ],
  globals: true,
},
```

Step 31 — Setup Testing Library

Tạo src/test/setup.js và import jest-dom.

```
import "@testing-library/jest-dom"
```

Step 32 — package.json scripts

Thêm scripts để chạy test ở watch mode hoặc chạy một lần.

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "vitest",
    "test:run": "vitest run"
  }
}
```

Step 33 — Test component

Component test nên kiểm tra UI user nhìn thấy thay vì implementation details.

```
import {
  render,
  screen,
} from "@testing-library/react"

test("shows projects title", () => {
  render(<ProjectsPage />)

  expect(
    screen.getByText("Projects")
  ).toBeInTheDocument()
})
```

Step 34 — Test API bằng mock

Mock api.post để test getProjects mà không cần Rails thật.

```
vi.spyOn(
  api,
  "post"
).mockResolvedValue({
  data: {
    data: {
      projects: {
        nodes: [],
      },
    },
  },
```

```
})

const data =
  await getProjects()

expect(
  data.projects.nodes
).toEqual([])
```

Step 35 — Chạy tests

npm test chạy watch mode; npm run test:run chạy một lần.

```
npm test
npm run test:run
```

Step 36 — Cấu trúc cuối cùng

Luồng chính cần nhớ: React component gọi API layer; API dùng graphqlRequest; Axios gửi /graphql tới Vite; Vite proxy forward sang Rails; Rails đi qua GraphqlController, schema, resolver, policy và database.

```
React Component
      ↓
getProjects()
      ↓
graphqlRequest()
      ↓
Axios
      ↓
POST /graphql
      ↓
localhost:5173
      ↓
Vite proxy
      ↓
127.0.0.1:3000/graphql
      ↓
Rails
      ↓
GraphqlController
      ↓
DevelopmentSchema
      ↓
Resolver
      ↓
Policy
      ↓
Database

Nguyên tắć:
Development:
React -> /graphql -> Vite proxy -> Rails

axios.js:
baseURL: ""

graphql.js:
api.post("/graphql")

vite.config.js:
```

```
"/graphql": {
  target: "http://127.0.0.1:3000"
}
```

Tóm tắt kiến trúc

Ba file phải khớp với nhau trong development: axios.js dùng baseURL rỗng; graphql.js gọi /graphql; vite.config.js proxy /graphql sang Rails.

```
// axios.js
baseURL: ""

// graphql.js
api.post("/graphql")

// vite.config.js
"/graphql": {
  target: "http://127.0.0.1:3000"
}
```

Checklist học viên

- Rails chạy đúng port.
- Vite chạy và frontend mở được.
- Axios không hard-code Rails URL khi dùng proxy.
- Network tab thấy localhost:5173/graphql.
- Không còn CORS.
- Nếu 502: kiểm tra Rails/proxy target.
- Nếu 422: kiểm tra CSRF/session.
- Devise session dùng withCredentials.
- Vitest + Testing Library chạy xanh.